



Department of Artificial Intelligence and Machine Learning

IV SEMESTER

LAB MANUAL

For

STATISTICAL PROGRAMMING LABORATORY

SUB CODE: AIL47

Credits: 0:0:1

Handled by

Dr. Meeradevi

Course Assessment and Evaluation:

Continuous Internal Evaluation (CIE): 50 Marks		
Assessment Tools	Marks	Course Outcomes (Cos) addressed
Lab Test-I	10	CO1, CO2
Lab Test-II	10	CO3
Weekly Evaluation – Lab Record	30	-
The Final CIE out of 50 Marks = Marks of Lab Record + Marks scored in Lab Test-I + Marks scored in Lab Test-II		
Semester End Examination (SEE)		
Course End Examination (One full question from the Lab Question Bank, Programs will be coded using C and executed)	50	CO1, CO2, CO3

Contents

Sl. No.	Program
PART A	
1	Write a R program to extract first 10 English letters in lower case and last 10 letters in upper case and extract letters between 22nd to 24th letters in upper case.
2	Suppose that <code>queue <- c("Steve", "Russell", "Alison", "Liam")</code> and that <code>queue</code> represents a supermarket queue with Steve first in line. Using R expressions update the supermarket queue as successively: a) Barry arrives; b) Steve is served; c) Pam talks her way to the front with one item; d) Barry gets impatient and leaves; e) Alison gets impatient and leaves. f) For the last case you should not assume that you know where in the queue, Alison is standing. g) Finally, using the function <code>which(x)</code> , find the position of Russell in the queue.
3	Plot the graph of linear equation, Non-linear equation (square, cubic, square root, etc.), Trigonometric function and logarithmic functions
4	The performance of a student in 3rd semester CSE as given below {SUB1, SUB2, SUB3, SUB4, SUB5, SUB6} with score {91, 73, 65, 45, 54, 32}. a) Draw the Bar chart to indicate the performance of the student. b) Draw the pie chart to indicate the performance of the student. c) Find the average marks and discuss qualitatively about the performance in each subjects.
5	Write a R program to create an array of two 3x3 matrices each with 3 rows and 3 columns from two given two vectors. Print the second row of the second matrix of the array and the element in the 3rd row and 3rd column of the 1st matrix.
6	Write a R program to demonstrate the use of <code>rbind()</code> function to combine two or more data frames in vertically.
7	Write a R program to demonstrate various Apply functions(<code>lapply</code> and <code>sapply</code>)
8	Write a R Program to demonstrate debugging through <code>traceback()</code> .
9	Write a R program to demonstrate <code>mutate()</code> , <code>select()</code> , <code>filter()</code> using <code>dplyr</code> package.
PART B	
1	You are working as a cashier at a grocery store. Your task is to create a program that simulates the checkout process for a customer's shopping cart. The program should calculate the total cost of the items, including tax, and provide a detailed receipt. i. Define a list of products, each represented as a dictionary with keys: "name", "price", and "quantity". ii. Allow the cashier to input the products in the customer's shopping cart, including the name, price, and quantity of each item. iii. Calculate the subtotal (price * quantity) for each item and display a detailed receipt with product names, quantities, prices, and subtotals. iv. Calculate the total cost of the items in the cart before tax. v. Apply a tax rate (e.g., 8%) to the total cost to calculate the tax amount. vi. Calculate the final total cost by adding the tax amount to the total cost before tax.
2	You have been tasked with creating a program that calculates and assigns grades for students enrolled in multiple courses. The program will take input for the marks obtained by 10 students

	in 5 different courses, compute the total and average marks for each student, and assign corresponding grades based on their average performance. Declare constants for the number of students (num_students) and the number of courses (num_courses). Initialize an empty list to store student information. For each student: • Input the student's name. • Input marks for each of the 5 courses. • Calculate the total marks and average marks. • Determine the grade based on the average marks using a grading scale. • Display the student information, including their name, individual course marks, total marks, average marks, and the assigned grade.
3	You are developing an inventory management system for a small store. The system needs to handle inventory items and their quantities. Write a program that uses arrays to store inventory items and their quantities, and includes functions to add new items, update quantities, and display the inventory. • Define an array to store inventory items. • Define an array to store corresponding quantities. • Implement functions to: ○ Add a new item along with its quantity. ○ Update the quantity of an existing item. ○ Display the inventory items and quantities. ○ Use the functions to manage the inventory and handle user interactions.
4	You are working as an educational analyst and need to analyze the performance of students in a school. You have data on student names, their scores in different subjects, and attendance. Write a program that uses data frames to manage and analyze student data, including calculating average scores, identifying students with low attendance, and generating a report. Create a data frame to store student information with columns: "Name", "Math_Score", "Science_Score", "History_Score", "Attendance". Implement functions to: • Calculate the average scores for each student. • Identify students with attendance below a certain threshold. • Generate a report with student names, average scores, and attendance status. • Use the functions to analyse student performance and generate the report.
5	You are a data analyst at a retail company that sells products online. The company is interested in predicting sales for the upcoming months to better manage inventory and plan marketing strategies. As part of your role, you need to develop a program that utilizes time series analysis to forecast sales based on a historical sales dataset. Write an R program to forecast sales for the next three months using time series analysis techniques. The program should perform the following steps: • Load the required libraries, including the forecast package. • Create a data frame with two columns: Month and Sales. The Month column should contain a sequence of dates from January 2023 to June 2023 (inclusive), and the Sales column should contain the corresponding sales amounts (12000, 15000, 18000, 16000, 20000, 22000). • Convert the sales data into a time series object with a monthly frequency. • Fit an ARIMA (AutoRegressive Integrated Moving Average) model to the sales time series using the auto.arima() function. • Forecast sales for the next three months using the fitted ARIMA model and the forecast() function. • Display the forecasted sales results, including point forecasts and prediction intervals.

6	You are a data analyst working for an e-commerce company that specializes in selling a variety of products online. The company aims to analyze customer purchase data comprehensively to gain insights into customer behavior and spending patterns. Your goal is to develop a R program that performs an in-depth analysis of customer purchase data. You will calculate various statistical measures and generate visualizations to understand the distribution of purchase amounts among customers. Note: Load the necessary libraries, including the dplyr and ggplot2 packages. Given the example customer purchase data provided below, create a data frame named purchase_data with two columns: CustomerID and PurchaseAmount. Calculate and display the following statistical measures: • Mean (average) purchase amount • Median purchase amount • Standard deviation of purchase amounts • 1st quartile (25th percentile) of purchase amounts • 3rd quartile (75th percentile) of purchase amounts Create a histogram to visualize the distribution of purchase amounts using the ggplot2 package. Display the histogram with appropriate labels and titles. Example Customer Purchase Data: <table border="1"> <thead> <tr> <th>CustomerID</th><th>PurchaseAmount</th></tr> </thead> <tbody> <tr> <td>101</td><td>150</td></tr> <tr> <td>102</td><td>200</td></tr> <tr> <td>103</td><td>120</td></tr> <tr> <td>104</td><td>300</td></tr> <tr> <td>105</td><td>80</td></tr> </tbody> </table>	CustomerID	PurchaseAmount	101	150	102	200	103	120	104	300	105	80
CustomerID	PurchaseAmount												
101	150												
102	200												
103	120												
104	300												
105	80												
7	Write an R program that generates two matrices, matrix_A and matrix_B, and conducts operations including element-wise addition, scalar multiplication, matrix transpose, and multiplication.												
8	You are a data analyst tasked with analyzing and visualizing a dataset. The dataset contains information about sales data. You need to create a program that generates various types of plots to help understand and present the data effectively. Write a program that performs data analysis and generates visualizations for a given dataset. The program should: - Load the necessary libraries (ggplot2). - Perform the following tasks: - Create a scatter plot to visualize the relationship between sales, engagement and category. - Create a line plot to display the trend of sales over time. - Generate a histogram to visualize the distribution of sales data. - Customize the appearance of each plot, such as color, labels, and titles. - Provide appropriate titles for each plot and the axes.												
9	You are a data analyst at an e-commerce company that sells a variety of products online. The company has provided you with a dataset containing information about customer purchases. Your task is to perform a comprehensive data analysis to gain insights into customer behavior and spending patterns. Dataset Description: The dataset customer_purchases.csv contains the following columns: CustomerID: Unique identifier for each customer. PurchaseAmount: The amount spent by the customer on a purchase.												

Problem Tasks:

You are required to perform the following tasks using R:

Task 1: Load the Dataset

Load the necessary libraries, including readr and dplyr.

Read the dataset customer_purchases.csv into a data frame named purchase_data.

Task 2: Data Summary

Calculate and display the total number of records in the dataset.

Calculate and display the total number of unique customers in the dataset.

Task 3: Calculate Statistical Measures

Calculate and display the mean (average) purchase amount.

Calculate and display the median purchase amount.

Calculate and display the standard deviation of purchase amounts.

Task 4: Customer Segmentation

Create a new column named Segment in the purchase_data data frame based on the following criteria:

"Low Spender" if the purchase amount is less than the median.

"High Spender" if the purchase amount is greater than or equal to the median.

Task 5: Visualize Data

PART A

Exercise 1: Write a R program to extract first 10 English letters in lower case and last 10 letters in upper case and extract letters between 22nd to 24th letters in upper case.

Solution:

```
print("First 10 letters in lower case:") t = head(letters, 10)
print(t)
print("Last 10 letters in upper case:") t = tail(LETTERS, 10)
print(t)
print("Letters between 22nd to 24th letters in upper case:") e = tail(LETTERS[22:24])
print(e)
```

Exercise 2: Suppose that `queue <- c("Steve", "Russell", "Alison", "Liam")` and that queue represents a supermarket queue with Steve first in line. Using R expressions update the supermarket queue as successively:

- a) Barry arrives;
- b) Steve is served;
- c) Pam talks her way to the front with one item;
- d) Barry gets impatient and leaves;
- e) Alison gets impatient and leaves.
- f) For the last case you should not assume that you know where in the queue, Alison is standing.
- g) Finally, using the function `which(x)`, find the position of Russell in the queue.

Solution:

```
#set queue
queue <- c("Steve", "Russell", "Alison", "Liam")
#add barry to the end of the queue
queue[length(queue)+1]="Barry"

#remove steve from the queue queue
<- queue[queue != "Steve"]

#add pam to the beginning of the queue queue <-
append(queue, "Pam", after = 0)#remove barry
from queue

queue <- queue[queue != "Barry"]
#remove alison from queue

queue <- queue[queue != "Alison"]
queue
```

```
#find russell's position in the queue
which(queue=="Russell")
```

Exercise 3: Plot the graph of linear equation, Non-linear equation (square, cubic, square root, etc.), Trigonometric function and logarithmic functions

Solution:

```
x <- seq(-5, 10, by = 0.1) M <- 2
```

```
C <- 4
```

```
eq <- M * x + C neq1 <- x^2 neq2 <- x^3
```

```
neq3 <- sqrt(abs(x))
```

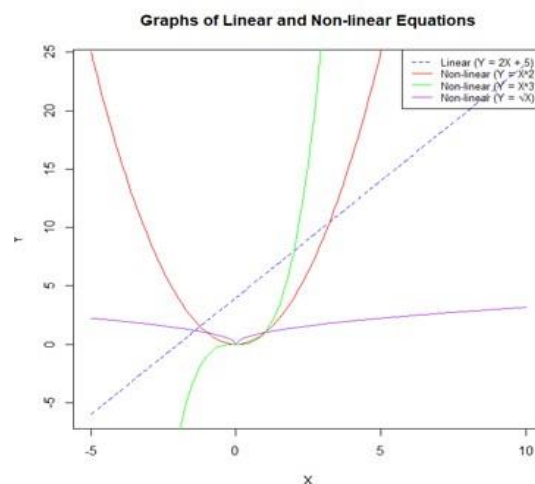
```
plot(x, eq, type = "l", col = "blue", lty = 2,
```

```
xlab = "X", ylab = "Y", main = "Graphs of Linear and Non-linear Equations") lines(x, neq1, col = "red")
```

```
lines(x, neq2, col = "green") lines(x, neq3, col = "purple")
```

```
legend("topright", legend = c("Linear (Y = 2X + 5)", "Non-linear (Y = X^2)", "Non-linear (Y = X^3)",
"Non-linear (Y = √X)"),
```

```
col = c("blue", "red", "green", "purple"), lty = c(2, 1, 1, 1), cex = 0.8)
```



Exercise 4: The performance of a student in 3rd semester CSE as given below {SUB1, SUB2, SUB3, SUB4, SUB5, SUB6} with score {91, 73, 65, 45, 54, 32}.

- Draw the Bar chart to indicate the performance of the student.
- Draw the pie chart to indicate the performance of the student.
- Find the average marks and discuss qualitatively about the performance in each subjects.

Solution:

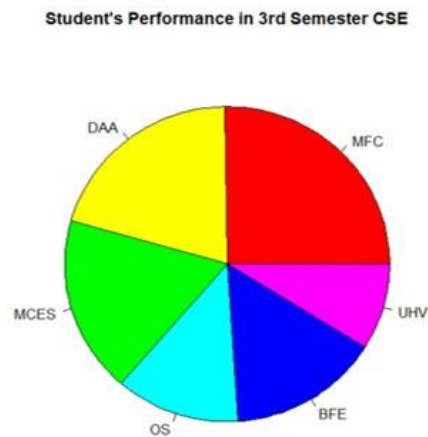
```
subject<- c("MFC", "DAA", "MCES"
```

```
, "OS", "BFE", "UHV")
```

```

marks <- c(91, 73, 65, 45, 54, 32)
avg<- mean(marks)
barplot(marks, names.arg = subject, col = "skyblue",
main = "Student's Performance i n 3rd Semester CSE",
xlab = "Subjects", ylab = "Marks
")
pie(marks, labels = subject, col = rain bow(length(marks)),
main = "Student's Performance in 3 rd Semester CSE")
cat("Average Marks:", avg, "\n") cat("Qualitative Performance:\n") for (i in 1:length(marks)) {
cat(subject[i], ":", ifelse(marks[i] >= avg, "Good", "Needs Improvement"), "\n")
}

```



Exercise 5: Write a R program to create an array of two 3x3 matrices each with 3 rows and 3 columns from two given two vectors. Print the second row of the second matrix of the array and the element in the 3rd row and 3rd column of the 1st matrix.

Solution:

```

print("Two vectors of different lengths:")
v1 = c(1,3,4,5)
v2 = c(10,11,12,13,14,15)
print(v1)
print(v2)
result = array(c(v1,v2),dim = c(3,3,2))
print("New array:")
print(result)
print("The second row of the second matrix of the array:")
print(result[2,,2])
print("The element in the 3rd row and 3rd column of the 1st matrix:")

```

```
print(result[3,3,1])
```

Output:

```
[1] "Two vectors of different lengths:"
```

```
[1] 1 3 4 5
```

```
[1] 10 11 12 13 14 15
```

```
[1] "New array:"
```

```
, , 1
```

```
      [,1] [,2] [,3]
```

```
[1,]      1      5     12
```

```
[2,]      3     10     13
```

```
[3,]      4     11     14
```

```
, , 2
```

```
      [,1] [,2] [,3]
```

```
[1,]     15      4      11
```

```
[2,]      1      5      12
```

```
[3,]      3     10      13
```

```
[1] "The second row of the second matrix of the array:"
```

```
[1] 1 5 12
```

```
[1] "The element in the 3rd row and 3rd column of the 1st matrix:"
```

```
[1] 14
```

Exercise 6: Write a R program to demonstrate the use of `rbind()` function to combine two or more data frames in vertically.

Solution:

```
Data_Frame1 <- data.frame (  
  Training = c("Strength", "Stamina", "Other"),  
  Pulse = c(100, 150, 120),  
  Duration = c(60, 30, 45)  
)  
  
Data_Frame2 <- data.frame (  
  Training = c("Stamina", "Stamina", "Strength"),  
  Pulse = c(140, 150, 160),  
  Duration = c(30, 30, 20)
```

```
)
New_Data_Frame <- rbind(Data_Frame1, Data_Frame2)
New_Data_Frame
Training Pulse Duration
1 Strength 100 60
2 Stamina 150 30
3 Other 120 45
4 Stamina 140 30
5 Stamina 150 30
6 Strength 160 20
```

Exercise 7: Write a R program to demonstrate various Apply functions(lapply and sapply)
Solution:

Syntax: lapply(x, fun)

Parameters:

x: determines the input vector or an object.

fun: determines the function that is to be applied to input data.

```
# create sample data
names <- c("priyank", "abhiraj", "pawananjani", "sudhanshu", "devraj")
print( "original data:")
names
# apply lapply() function
print("data after lapply():")
lapply(names, toupper)
```

Output:

```
[1] "original data:"
'priyank' 'abhiraj' 'pawananjani' 'sudhanshu' 'devraj'

[1] "data after lapply():"
1. 'PRIYANK'
2. 'ABHIRAJ'
3. 'PAWANANJANI'
4. 'SUDHANSHU'
5. 'DEVRAJ'
```

Syntax: sapply(x, fun)

Parameters:

x: determines the input vector or an object.

fun: determines the function that is to be applied to input data.

create sample data

```
sample_data<- data.frame( x=c(1,2,3,4,5,6),y=c(3,2,4,2,34,5))
```

```
print( "original data:")
```

```
sample_data
```

apply sapply() function

```
print("data after sapply():")
```

```
sapply(sample_data, max)
```

Output:

```
[1] "original data:"
```

x	y
1	3
2	2
3	4
4	2
5	34
6	5

```
[1] "data after sapply():"
```

x	6
y	34

Exercise 8: Write a R Program to demonstrate debugging through traceback ().

Solution:

Function 1

```
function_1 <- function(a){
```

```
a + 5
```

```
}
```

Function 2

```
function_2 <- function(b){
```

```
function_1(b)
```

```
}
```

```
# Calling error handler
options(error = traceback)

function_2("s")
```

Output:

Error in a + 5 : non-numeric argument to binary operator

2: function_1(b) at #1

1: function_2("s")

Exercise 9: Write a R program to demonstrate mutate(), select(), filter() using dplyr package.

```
i. # import dplyr package
library(dplyr)

# create a data frame
stats <- data.frame(player=c('A', 'B', 'C', 'D'),
                    runs=c(100, 200, 408, 19),
                    wickets=c(17, 20, 7, 5))
```

```
# add new column avg
mutate(stats, avg=runs/4)
```

```
# drop all and create a new column
transmute(stats, avg=runs/4)
```

```
ii. # import dplyr package
library(dplyr)
```

```
# create a data frame
stats <- data.frame(player=c('A', 'B', 'C', 'D'),
                    runs=c(100, 200, 408, 19),
                    wickets=c(17, 20, NA, 5))
```

```
# fetch required column data
select(stats, player, wickets)
```

```
iii. # import dplyr package
library(dplyr)
```

```
# create a data frame
stats <- data.frame(player=c('A', 'B', 'C', 'D'),
                    runs=c(100, 200, 408, 19),
                    wickets=c(17, 20, NA, 5))
```

```
# fetch players who scored more
# than 100 runs
filter(stats, runs>100)
```

PART B

Exercise 1: Basic Arithmetic Operations

You are working as a cashier at a grocery store. Your task is to create a program that simulates the checkout process for a customer's shopping cart. The program should calculate the total cost of the items, including tax, and provide a detailed receipt.

- i. Define a list of products, each represented as a dictionary with keys: "name", "price", and "quantity".
- ii. Allow the cashier to input the products in the customer's shopping cart, including the name, price, and quantity of each item.
- iii. Calculate the subtotal (price * quantity) for each item and display a detailed receipt with product names, quantities, prices, and subtotals.
- iv. Calculate the total cost of the items in the cart before tax.
- v. Apply a tax rate (e.g., 8%) to the total cost to calculate the tax amount.
- vi. Calculate the final total cost by adding the tax amount to the total cost before tax.

Code in R: # List of products

List of products

```
products <- list(  
  list(name = "Apple", price = 0.5),  
  list(name = "Banana", price = 0.3),  
  list(name = "Milk", price = 2),  
  list(name = "Bread", price = 1.5),  
  list(name = "Eggs", price = 2.5)  
)
```

Initialize shopping cart as an empty list

```
shopping_cart <- list()
```

Define items to be added to the cart

```
cart_items_to_add <- list(  
  list(name = "Apple", quantity = 3),  
  list(name = "Milk", quantity = 2)  
)
```

Add items to the shopping cart

```
for (item in cart_items_to_add) {  
  product_name <- item$name
```

```
quantity <- item$quantity
```

```
# Find the product in the list
```

```
product <- NULL
```

```
for (p in products) {
```

```
  if (p$name == product_name) {
```

```
    product <- p
```

```
    break
```

```
  }
```

```
}
```

```
if (!is.null(product)) {
```

```
  cart_item <- list(name = product$name, price = product$price, quantity = quantity)
```

```
  shopping_cart <- c(shopping_cart, list(cart_item))
```

```
  cat("Item added to cart.\n")
```

```
} else {
```

```
  cat("Product not found.\n")
```

```
}
```

```
}
```

```
# Calculate and display receipt
```

```
subtotal <- 0
```

```
cat("\nReceipt:\n")
```

```
for (item in shopping_cart) {
```

```
  item_subtotal <- item$price * item$quantity
```

```
  cat(sprintf("%s (%d units) - Price: $%.2f - Subtotal: $%.2f\n", item$name, item$quantity, item$price, item_subtotal))
```

```
  subtotal <- subtotal + item_subtotal
```

```
}
```

```
tax_rate <- 0.08
```

```
tax_amount <- subtotal * tax_rate
```

```
total_cost_before_tax <- subtotal
```

```
total_cost <- total_cost_before_tax + tax_amount
```

```
cat("\nSubtotal: $%.2f\n", subtotal)
```

```
cat("Tax Amount (8%): $%.2f\n", tax_amount)
```

```
cat("Total Cost: $%.2f\n", total_cost)
```

Exercise 2: Loops Operations

You have been tasked with creating a program that calculates and assigns grades for students enrolled in multiple courses. The program will take input for the marks obtained by 10 students in 5 different courses, compute the total and average marks for each student, and assign corresponding grades based on their average performance.

Declare constants for the number of students (`num_students`) and the number of courses (`num_courses`).

Initialize an empty list to store student information.

For each student:

- Input the student's name.
- Input marks for each of the 5 courses.
- Calculate the total marks and average marks.
- Determine the grade based on the average marks using a grading scale.
- Display the student information, including their name, individual course marks, total marks, average marks, and the assigned grade.

Program:

```
# Constants
```

```
num_students <- 5
```

```
num_courses <- 5
```

```
# Predefined student names
```

```
student_names <- c("Arun Rahul", "Bheem Kumar", "Raj jumar", "Jahal A R", "Suresh")
```

```
# Predefined course marks for each student
```

```
course_marks <- matrix(c(
```

```
  85, 92, 78, 88, 95,
```

```
  75, 80, 85, 70, 60,
```

```
  100, 78, 56, 34, 56,
```

```
  78, 45, 67, 89, 90,
```

```
  89, 80, 67, 78, 90
```

```
), nrow = num_students, byrow = TRUE)
```

```
# Initialize a list to store student information
```

```
student_records <- list()
```

```
# Loop for each student
```

```

for (student_index in 1:num_students) {
  student_name <- student_names[student_index]

  # Initialize variables for calculations
  total_marks <- sum(course_marks[student_index, ])
  average_marks <- total_marks / num_courses

  # Determine grade based on average marks
  grade <- ifelse(average_marks >= 90, "A",
                 ifelse(average_marks >= 80, "B",
                        ifelse(average_marks >= 70, "C",
                               ifelse(average_marks >= 60, "D", "F"))))

  # Store student information in a record
  student_record <- list(name = student_name, marks = course_marks[student_index, ],
                        total = total_marks, average = average_marks, grade = grade)

  student_records <- c(student_records, list(student_record))
}

# Display student information
cat("\nStudent Grade Report:\n")
for (student_record in student_records) {
  cat("\nName:", student_record$name, "\n")
  cat("Marks:", student_record$marks, "\n")
  cat("Total Marks:", student_record$total, "\n")
  cat("Average Marks:", student_record$average, "\n")
  cat("Grade:", student_record$grade, "\n")
}

```

Output of the program:

Student Grade Report:

Name: Arun Rahul

Marks: 85 92 78 88 95

Total Marks: 438

Average Marks: 87.6

Grade: B

Name: Bheem Kumar

Marks: 75 80 85 70 60

Total Marks: 370

Average Marks: 74

Grade: C

Name: Raj jumar

Marks: 100 78 56 34 56

Total Marks: 324

Average Marks: 64.8

Grade: D

Name: Jahal A R

Marks: 78 45 67 89 90

Total Marks: 369

Average Marks: 73.8

Grade: C

Name: Suresh

Marks: 89 80 67 78 90

Total Marks: 404

Average Marks: 80.8

Grade: B

Exercise 3: arrays and Functions:

You are developing an inventory management system for a small store. The system needs to handle inventory items and their quantities. Write a program that uses arrays to store inventory items and their quantities, and includes functions to add new items, update quantities, and display the inventory.

- Define an array to store inventory items.
- Define an array to store corresponding quantities.
- Implement functions to:
 - Add a new item along with its quantity.
 - Update the quantity of an existing item.
 - Display the inventory items and quantities.
 - Use the functions to manage the inventory and handle user interactions.

Program:

```
# Initialize arrays for inventory items and quantities
```

```
inventory_items <- character(0)
```

```
inventory_quantities <- numeric(0)
```

```
# Function to add a new item with quantity
```

```
add_item <- function(item, quantity) {
```

```
  inventory_items <- c(inventory_items, item)
```

```
  inventory_quantities <- c(inventory_quantities, quantity)
```

```
  cat("Item added to inventory.\n")
```

```
}
```

```
# Function to update quantity of an existing item
```

```
update_quantity <- function(item, new_quantity) {
```

```
  if (item %in% inventory_items) {
```

```
    item_index <- which(inventory_items == item)
```

```
    inventory_quantities[item_index] <- new_quantity
```

```
    cat("Quantity updated.\n")
```

```
  } else {
```

```
    cat("Item not found in inventory.\n")
```

```
  }
```

```
}
```

```
# Function to display inventory
```

```
display_inventory <- function() {
```

```

cat("Inventory Items and Quantities:\n")
for (i in 1:length(inventory_items)) {
  cat(sprintf("%s: %d\n", inventory_items[i], inventory_quantities[i]))
}
}

# Main program
while (TRUE) {
  cat("\n1. Add Item\n2. Update Quantity\n3. Display Inventory\n4. Exit\n")
  choice <- as.integer(readline("Enter your choice: "))

  if (choice == 1) {
    item <- readline("Enter item name: ")
    quantity <- as.integer(readline("Enter quantity: "))
    add_item(item, quantity)
  } else if (choice == 2) {
    item <- readline("Enter item name: ")
    new_quantity <- as.integer(readline("Enter new quantity: "))
    update_quantity(item, new_quantity)
  } else if (choice == 3) {
    display_inventory()
  } else if (choice == 4) {
    cat("Exiting the program. Goodbye!\n")
    break
  } else {
    cat("Invalid choice. Please try again.\n")
  }
}

```

Output:

```

1. Add Item
2. Update Quantity
3. Display Inventory
4. Exit
Enter your choice: 1
Enter item name: rice

```

Enter quantity: 100
Item added to inventory.

1. Add Item
2. Update Quantity
3. Display Inventory
4. Exit

Enter your choice: 1
Enter item name: bicults
Enter quantity: 23
Item added to inventory.

1. Add Item
2. Update Quantity
3. Display Inventory
4. Exit

Enter your choice: 1
Enter item name: ragi
Enter quantity: 78
Item added to inventory.

1. Add Item
2. Update Quantity
3. Display Inventory
4. Exit

Enter your choice: 3
Inventory Items and Quantities:
rice: 100
bicults: 23
ragi: 78

1. Add Item
2. Update Quantity
3. Display Inventory
4. Exit

Enter your choice: 2
Enter item name: ragi
Enter new quantity: 90
Quantity updated.

1. Add Item
2. Update Quantity
3. Display Inventory
4. Exit

Enter your choice: 3
Inventory Items and Quantities:
rice: 100
bicults: 23
ragi: 90

1. Add Item
2. Update Quantity
3. Display Inventory
4. Exit

Enter your choice:

Exercise 4: Dataframe

You are working as an educational analyst and need to analyze the performance of students in a school. You have data on student names, their scores in different subjects, and attendance. Write a program that uses data frames to manage and analyze student data, including calculating average scores, identifying students with low attendance, and generating a report.

Create a data frame to store student information with columns: "Name", "Math_Score", "Science_Score", "History_Score", "Attendance".

Implement functions to:

- Calculate the average scores for each student.
- Identify students with attendance below a certain threshold.
- Generate a report with student names, average scores, and attendance status.
- Use the functions to analyse student performance and generate the report.

Program:

```
# Load the 'dplyr' package for data manipulation
```

```
library(dplyr)
```

```
# Create a data frame to store student information
```

```
student_data <- data.frame(
```

```
  Name = character(0),
```

```
  Math_Score = numeric(0),
```

```
  Science_Score = numeric(0),
```

```
  History_Score = numeric(0),
```

```
  Attendance = numeric(0)
```

```
)
```

```
# Function to add student information
```

```
add_student <- function(name, math_score, science_score, history_score, attendance) {
```

```
  new_student <- data.frame(
```

```
    Name = name,
```

```
    Math_Score = math_score,
```

```
    Science_Score = science_score,
```

```
    History_Score = history_score,
```

```
    Attendance = attendance
```

```
)
```

```
student_data <-< bind_rows(student_data, new_student)
```

```
cat("Student information added.\n")
```

```
}
```

```
# Function to calculate average scores
```

```
calculate_average_scores <- function() {
```

```
  avg_scores <- student_data %>%
```

```
    mutate(Average_Score = (Math_Score + Science_Score + History_Score) / 3) %>%
```

```
    select(Name, Average_Score)
```

```
  return(avg_scores)
```

```
}
```

```
# Function to identify students with low attendance
```

```
identify_low_attendance <- function(threshold) {
```

```
  low_attendance <- student_data %>%
```

```
    filter(Attendance < threshold) %>%
```

```
    select(Name, Attendance)
```

```
  return(low_attendance)
```

```
}
```

```
# Function to generate a performance report
```

```
generate_report <- function() {
```

```
  avg_scores <- calculate_average_scores()
```

```
  low_attendance <- identify_low_attendance(70)
```

```
  report <- merge(avg_scores, low_attendance, by = "Name", all = TRUE)
```

```
  report$Attendance[is.na(report$Attendance)] <- 100
```

```
  cat("Performance Report:\n")
```

```
  print(report)
```

```
}
```

```
# Main program
```

```
while (TRUE) {
```

```
  cat("\n1. Add Student\n2. Generate Report\n3. Exit\n")
```

```
  choice <- as.integer(readline("Enter your choice: "))
```

```

if (choice == 1) {
  name <- readline("Enter student name: ")
  math_score <- as.numeric(readline("Enter math score: "))
  science_score <- as.numeric(readline("Enter science score: "))
  history_score <- as.numeric(readline("Enter history score: "))
  attendance <- as.numeric(readline("Enter attendance percentage: "))
  add_student(name, math_score, science_score, history_score, attendance)
} else if (choice == 2) {
  generate_report()
} else if (choice == 3) {
  cat("Exiting the program. Goodbye!\n")
  break
} else {
  cat("Invalid choice. Please try again.\n")
}
}

```

Explanation:

Load the 'dplyr' package for data manipulation

```
library(dplyr)
```

This line loads the 'dplyr' package, which provides a wide range of functions for data manipulation and transformation. It's used in this program for tasks like filtering, summarizing, and selecting data within data frames.

Create a data frame to store student information

```

student_data <- data.frame(
  Name = character(0),
  Math_Score = numeric(0),
  Science_Score = numeric(0),
  History_Score = numeric(0),
  Attendance = numeric(0)
)

```

Here, a data frame named `student_data` is created. It's initialized with empty columns for student names, math scores, science scores, history scores, and attendance. This data frame will hold all the student information.

```
# Function to add student information
```

```
add_student <- function(name, math_score, science_score, history_score, attendance) {  
  new_student <- data.frame(  
    Name = name,  
    Math_Score = math_score,  
    Science_Score = science_score,  
    History_Score = history_score,  
    Attendance = attendance  
  )  
  student_data <-<- bind_rows(student_data, new_student)  
  cat("Student information added.\n")  
}
```

This function `add_student` adds a new student's information to the `student_data` data frame. It creates a new data frame `new_student` with the provided information and then uses the `bind_rows` function from the 'dplyr' package to append this new student's data to the existing data frame.

```
# Function to calculate average scores
```

```
calculate_average_scores <- function() {  
  avg_scores <- student_data %>%  
    mutate(Average_Score = (Math_Score + Science_Score + History_Score) / 3) %>%  
    select(Name, Average_Score)  
  return(avg_scores)  
}
```

The `calculate_average_scores` function calculates the average scores for each student. It uses the `%>%` (pipe) operator from the 'dplyr' package to perform operations sequentially. The `mutate` function adds a new column `Average_Score` calculated by averaging math, science, and history scores. The `select` function extracts only the "Name" and "Average_Score" columns.

```
# Function to identify students with low attendance
```

```
identify_low_attendance <- function(threshold) {  
  low_attendance <- student_data %>%  
    filter(Attendance < threshold) %>%
```

```

    select(Name, Attendance)

return(low_attendance)

}

```

The `identify_low_attendance` function identifies students with attendance below a specified threshold. It uses the `filter` function to extract rows where attendance is below the threshold, and the `select` function to extract the "Name" and "Attendance" columns.

```

# Function to generate a performance report

generate_report <- function() {
  avg_scores <- calculate_average_scores()
  low_attendance <- identify_low_attendance(70)

  report <- merge(avg_scores, low_attendance, by = "Name", all = TRUE)
  report$Attendance[is.na(report$Attendance)] <- 100

  cat("Performance Report:\n")
  print(report)
}

```

The `generate_report` function generates a performance report. It calls the `calculate_average_scores` and `identify_low_attendance` functions to get average scores and low attendance data. The `merge` function combines the data based on the "Name" column. It also handles missing attendance values by replacing them with 100 using `is.na` and assignment.

Output:

```

1. Add Student
2. Generate Report
3. Exit
Enter your choice: 1
Enter student name: ramesh
Enter math score: 67
Enter science score: 89
Enter history score: 89
Enter attendance percentage: 80
Student information added.

1. Add Student
2. Generate Report
3. Exit
Enter your choice: 1
Enter student name: rajesh
Enter math score: 90
Enter science score: 78
Enter history score: 89
Enter attendance percentage: 90
Student information added.

```

1. Add Student
2. Generate Report
3. Exit

Enter your choice: 1

Enter student name: Roopesh

Enter math score: 98

Enter science score: 89

Enter history score: 98

Enter attendance percentage: 90

Student information added.

1. Add Student
2. Generate Report
3. Exit

Enter your choice: 2

Performance Report:

	Name	Average_Score	Attendance
1	rajesh	85.66667	100
2	ramesh	81.66667	100
3	Roopesh	95.00000	100

1. Add Student
2. Generate Report
3. Exit

Enter your choice: 1

Enter student name: Raj

Enter math score: 34

Enter science score: 23

Enter history score: 56

Enter attendance percentage: 35

Student information added.

1. Add Student
2. Generate Report
3. Exit

Enter your choice: 2

Performance Report:

	Name	Average_Score	Attendance
1	Raj	37.66667	35
2	rajesh	85.66667	100
3	ramesh	81.66667	100
4	Roopesh	95.00000	100

1. Add Student
2. Generate Report
3. Exit

Enter your choice:

Exercise 5: ARIMA

You are a data analyst at a retail company that sells products online. The company is interested in predicting sales for the upcoming months to better manage inventory and plan marketing strategies. As part of your role, you need to develop a program that utilizes time series analysis to forecast sales based on a historical sales dataset.

Write an R program to forecast sales for the next three months using time series analysis techniques. The program should perform the following steps:

- Load the required libraries, including the forecast package.
- Create a data frame with two columns: Month and Sales. The Month column should contain a sequence of dates from January 2023 to June 2023 (inclusive), and the Sales column should contain the corresponding sales amounts (12000, 15000, 18000, 16000, 20000, 22000).
- Convert the sales data into a time series object with a monthly frequency.
- Fit an ARIMA (AutoRegressive Integrated Moving Average) model to the sales time series using the `auto.arima()` function.
- Forecast sales for the next three months using the fitted ARIMA model and the `forecast()` function.
- Display the forecasted sales results, including point forecasts and prediction intervals.

Code:

```
# Load required libraries
```

```
library(forecast)
```

```
# Create a data frame with Month and Sales columns
```

```
sales_data <- data.frame(
```

```
  Month = seq(as.Date("2023-01-01"), as.Date("2023-06-01"), by = "months"),
```

```
  Sales = c(12000, 15000, 18000, 16000, 20000, 22000)
```

```
)
```

```
# Convert to time series
```

```
sales_ts <- ts(sales_data$Sales, frequency = 12)
```

```
# Fit ARIMA model
```

```
arima_model <- auto.arima(sales_ts)
```

```
# Forecast sales for next 3 months
```

```
forecast_result <- forecast(arima_model, h = 3)
```

```
# Display forecast results
```

```
print(forecast_result)
```

Output:

```
print(forecast_result)
```

	Point Forecast	Lo 80	Hi 80
Jul 1	22000	18285.70	25714.30
Aug 1	22000	16747.19	27252.81
Sep 1	22000	15566.65	28433.35

	Lo 95	Hi 95
Jul 1	16319.47	27680.53
Aug 1	13966.52	30033.48
Sep 1	12161.04	31838.96

Exercise 6: Customer Purchase Analysis for E-commerce Company (Enhanced)

You are a data analyst working for an e-commerce company that specializes in selling a variety of products online. The company aims to analyze customer purchase data comprehensively to gain insights into customer behavior and spending patterns.

Your goal is to develop an R program that performs an in-depth analysis of customer purchase data. You will calculate various statistical measures and generate visualizations to understand the distribution of purchase amounts among customers.

Note: Load the necessary libraries, including the dplyr and ggplot2 packages.

Given the example customer purchase data provided below, create a data frame named `purchase_data` with two columns: `CustomerID` and `PurchaseAmount`.

Calculate and display the following statistical measures:

- Mean (average) purchase amount
- Median purchase amount
- Standard deviation of purchase amounts
- 1st quartile (25th percentile) of purchase amounts
- 3rd quartile (75th percentile) of purchase amounts

Create a histogram to visualize the distribution of purchase amounts using the ggplot2 package. Display the histogram with appropriate labels and titles.

Example Customer Purchase Data:

CustomerID	PurchaseAmount
101	150
102	200
103	120
104	300
105	80

Solution:

```
# Load required libraries
```

```
library(dplyr)
```

```
library(ggplot2)
```

```
# Example customer purchase data
```

```
purchase_data <- data.frame(
```

```
  CustomerID = c(101, 102, 103, 104, 105),
```

```
  PurchaseAmount = c(150, 200, 120, 300, 80)
```

```
)
```

```
# Calculate statistical measures
```

```
mean_purchase <- mean(purchase_data$PurchaseAmount)
```

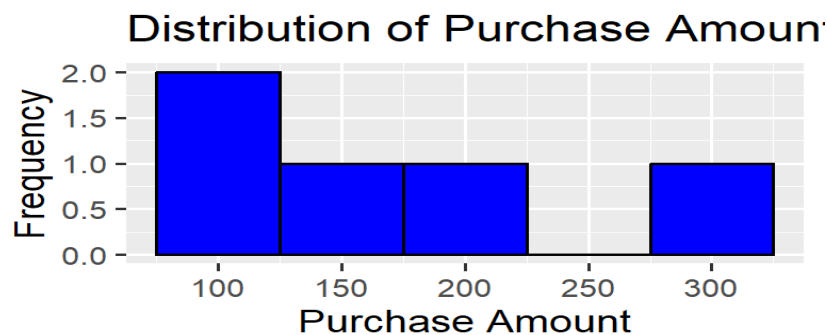
```

median_purchase <- median(purchase_data$PurchaseAmount)
sd_purchase <- sd(purchase_data$PurchaseAmount)
q1_purchase <- quantile(purchase_data$PurchaseAmount, probs = 0.25)
q3_purchase <- quantile(purchase_data$PurchaseAmount, probs = 0.75)

# Display results
cat("Mean Purchase Amount:", mean_purchase, "\n")
cat("Median Purchase Amount:", median_purchase, "\n")
cat("Standard Deviation of Purchase Amounts:", sd_purchase, "\n")
cat("1st Quartile of Purchase Amounts:", q1_purchase, "\n")
cat("3rd Quartile of Purchase Amounts:", q3_purchase, "\n")

# Create a histogram
ggplot(purchase_data, aes(x = PurchaseAmount)) +
  geom_histogram(binwidth = 50, fill = "blue", color = "black") +
  labs(title = "Distribution of Purchase Amounts", x = "Purchase Amount", y = "Frequency")

```



Exercise 7: Matrix Manipulation in R

Write an R program that generates two matrices, `matrix_A` and `matrix_B`, and conducts operations including element-wise addition, scalar multiplication, matrix transpose, and multiplication.

Code:

```
# Task 1: Matrix Creation
```

```
matrix_A <- matrix(c(1, 2, 3, 4, 5, 6, 7, 8, 9), nrow = 3, ncol = 3, byrow = TRUE)
```

```
matrix_B <- matrix(c(9, 8, 7, 6, 5, 4, 3, 2, 1), nrow = 3, ncol = 3, byrow = TRUE)
```

```
# Task 2: Matrix Manipulation
```

```
sum_matrix <- matrix_A + matrix_B
```

```
scaled_matrix <- matrix_A * 2
```

```
# Task 3: Matrix Operations
```

```
transposed_A <- t(matrix_A)
```

```
product_matrix <- matrix_A %*% matrix_B
```

```
# Task 4: Matrix Statistics
```

```
sum_matrix_A <- sum(matrix_A)
```

```
mean_matrix_B <- mean(matrix_B)
```

```
sd_matrix_B <- sd(matrix_B)
```

```
# Task 5: Visualization
```

```
library(ggplot2)
```

```
library(reshape2)
```

```
# Create a heatmap of matrix_A
```

```
heatmap_data <- melt(matrix_A)
```

```
heatmap_plot <- ggplot(heatmap_data, aes(x = Var1, y = Var2, fill = value)) +
```

```
  geom_tile() +
```

```
  scale_fill_gradient(low = "white", high = "blue") +
```

```
  labs(title = "Heatmap of Matrix A", x = "Column", y = "Row")
```

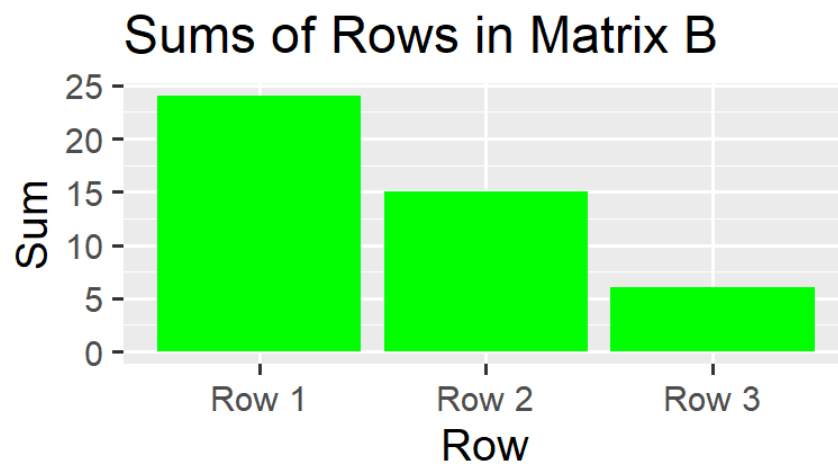
```
# Create a bar plot comparing sums of rows in matrix_B
```

```
row_sums <- rowSums(matrix_B)
```

```
row_names <- paste("Row", 1:3)
barplot_data <- data.frame(Row = row_names, Sum = row_sums)
barplot_plot <- ggplot(barplot_data, aes(x = Row, y = Sum)) +
  geom_bar(stat = "identity", fill = "green") +
  labs(title = "Sums of Rows in Matrix B", x = "Row", y = "Sum")

# Display the visualizations
print(heatmap_plot)
print(barplot_plot)
```

output:



Exercise 8: Visualization

You are a data analyst tasked with analyzing and visualizing a dataset. The dataset contains information about student performance in a course. You need to create a program that generates various types of plots to help understand and present the data effectively.

Write a program that performs data analysis and generates visualizations for a given dataset. The program should:

- a) Load the necessary libraries (ggplot2).
- b) Prepare example data with columns for student names, scores, and attendance percentages.
- c) Perform the following tasks:
 - i. Create a scatter plot to visualize the relationship between scores and attendance percentages.
 - ii. Generate a bar plot to show the distribution of scores among different students.
 - iii. Create a line plot to display the trend of scores over time (assuming different students' scores were collected at different time intervals).
 - iv. Generate a histogram to visualize the distribution of scores.
- d) Customize the appearance of each plot, such as color, labels, and titles.
- e) Arrange the plots in a way that they are easy to compare and understand.
- f) Provide appropriate titles for each plot and the axes.

Scatter Plot:

```
library(ggplot2)

# Example data
data <- data.frame(x = rnorm(50), y = rnorm(50))

# Scatter plot
ggplot(data, aes(x, y)) +
  geom_point() +
  labs(title = "Scatter Plot", x = "X-axis", y = "Y-axis")
```

Bar Plot:

```
library(ggplot2)

# Example data
data <- data.frame(category = c("A", "B", "C"), value = c(10, 20, 15))

# Bar plot
ggplot(data, aes(x = category, y = value)) +
  geom_bar(stat = "identity") +
  labs(title = "Bar Plot", x = "Category", y = "Value")
```

Line Plot:

```
library(ggplot2)

# Example data
data <- data.frame(x = 1:10, y = 2 * (1:10))

# Line plot
ggplot(data, aes(x, y)) +
  geom_line() +
  labs(title = "Line Plot", x = "X-axis", y = "Y-axis")
```

Histogram:

```
library(ggplot2)

# Example data
data <- data.frame(values = rnorm(100))

# Histogram
ggplot(data, aes(values)) +
  geom_histogram(binwidth = 0.5, fill = "blue", color = "black") +
  labs(title = "Histogram", x = "Values", y = "Frequency")
```

scatter, bar, line, histogram, and box plots

Load required libraries

```
library(ggplot2)
library(patchwork)
library(plotly)
```

Example data

```
set.seed(123)

data <- data.frame(
  Date = seq(as.Date("2023-01-01"), as.Date("2023-12-31"), by = "months"),
  Sales = cumsum(sample(5000:10000, 12, replace = TRUE)),
  Engagement = sample(1:10, 12, replace = TRUE),
  Category = rep(c("Electronics", "Clothing", "Books"), each = 4)
)
```

Scatter plot

```

scatter_plot <- ggplot(data, aes(x = Sales, y = Engagement, color = Category)) +
  geom_point() +
  labs(title = "Sales vs. Customer Engagement", x = "Sales", y = "Engagement")

# Bar plot for sales by category
bar_plot <- ggplot(data, aes(x = Category, y = Sales, fill = Category)) +
  geom_bar(stat = "identity") +
  labs(title = "Sales by Category", x = "Category", y = "Sales") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

# Line plot for sales over time
line_plot <- ggplot(data, aes(x = Date, y = Sales, color = Category)) +
  geom_line() +
  labs(title = "Sales Trend Over Time", x = "Date", y = "Sales")

# Histogram for customer engagement
histogram_plot <- ggplot(data, aes(Engagement)) +
  geom_histogram(binwidth = 1, fill = "green", color = "black") +
  labs(title = "Customer Engagement Distribution", x = "Engagement", y = "Frequency")

# Box plot for sales by category
box_plot <- ggplot(data, aes(Category, Sales, fill = Category)) +
  geom_boxplot() +
  labs(title = "Sales Distribution by Category", x = "Category", y = "Sales")

# Combine plots using patchwork
combined_plots <- scatter_plot +
  bar_plot +
  line_plot +
  histogram_plot +
  box_plot

# Display combined plots
print(combined_plots)

```

PIE chart in R:

Load required libraries

```
library(ggplot2)
```

Example data

```
data <- data.frame(  
  Category = c("Electronics", "Clothing", "Books"),  
  Sales = c(25000, 18000, 12000)  
)
```

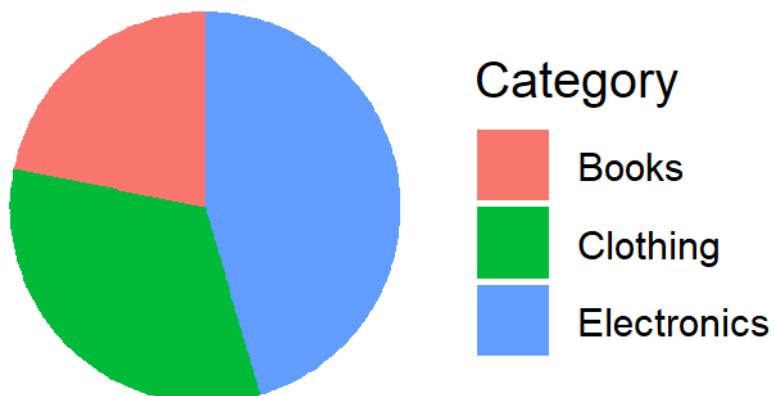
Create a pie chart

```
pie_chart <- ggplot(data, aes(x = "", y = Sales, fill = Category)) +  
  geom_bar(stat = "identity", width = 1) +  
  coord_polar("y", start = 0) + # Convert to polar coordinates  
  labs(title = "Pie Chart of Sales by Category") +  
  theme_void() # Remove axes and labels
```

Display the pie chart

```
print(pie_chart)
```

Pie Chart of Sales by Category



Exercise 9: Data Analysis: Statement: Customer Purchase Analysis

You are a data analyst at an e-commerce company that sells a variety of products online. The company has provided you with a dataset containing information about customer purchases. Your task is to perform a comprehensive data analysis to gain insights into customer behavior and spending patterns.

Dataset Description:

The dataset `customer_purchases.csv` contains the following columns:

CustomerID: Unique identifier for each customer.

PurchaseAmount: The amount spent by the customer on a purchase.

Problem Tasks:

You are required to perform the following tasks using R:

Task 1: Load the Dataset

Load the necessary libraries, including `readr` and `dplyr`.

Read the dataset `customer_purchases.csv` into a data frame named `purchase_data`.

Task 2: Data Summary

Calculate and display the total number of records in the dataset.

Calculate and display the total number of unique customers in the dataset.

Task 3: Calculate Statistical Measures

Calculate and display the mean (average) purchase amount.

Calculate and display the median purchase amount.

Calculate and display the standard deviation of purchase amounts.

Task 4: Customer Segmentation

Create a new column named `Segment` in the `purchase_data` data frame based on the following criteria:

"Low Spender" if the purchase amount is less than the median.

"High Spender" if the purchase amount is greater than or equal to the median.

Task 5: Visualize Data

Create a histogram to visualize the distribution of purchase amounts using the `ggplot2` package. Customize the plot with appropriate labels and titles.

Code:

```
library(readr)
library(dplyr)
library(ggplot2)
```

```
# Task 1: Load the Dataset
```

```
purchase_data <- read_csv("D:/R Lab 2024/customer_purchases.csv")
```

```
# Load required libraries
```

```

# Task 2: Data Summary
total_records <- nrow(purchase_data)
total_customers <- n_distinct(purchase_data$CustomerID)

cat("Total Number of Records:", total_records, "\n")
cat("Total Number of Unique Customers:", total_customers, "\n")

# Task 3: Calculate Statistical Measures
mean_purchase <- mean(purchase_data$PurchaseAmount)
median_purchase <- median(purchase_data$PurchaseAmount)
sd_purchase <- sd(purchase_data$PurchaseAmount)

cat("Mean Purchase Amount:", mean_purchase, "\n")
cat("Median Purchase Amount:", median_purchase, "\n")
cat("Standard Deviation of Purchase Amounts:", sd_purchase, "\n")

# Task 4: Customer Segmentation (Based on Quartiles)
q1_purchase <- quantile(purchase_data$PurchaseAmount, probs = 0.25)
q2_purchase <- quantile(purchase_data$PurchaseAmount, probs = 0.5)
q3_purchase <- quantile(purchase_data$PurchaseAmount, probs = 0.75)

cat(q1_purchase, "\n")
cat(q2_purchase, "\n")
cat(q3_purchase, "\n")

purchase_data <- purchase_data %>%
  mutate(Segment = case_when(
    PurchaseAmount < q1_purchase ~ "Low Spender",
    PurchaseAmount >= q1_purchase & PurchaseAmount < q3_purchase ~ "Medium Spender",
    PurchaseAmount >= q3_purchase ~ "High Spender"
  ))

print(purchase_data)

# Task 5: Visualize Data (Histogram)
ggplot(purchase_data, aes(x = PurchaseAmount)) +
  geom_histogram(binwidth = 50, fill = "blue", color = "black") +
  labs(title = "Distribution of Purchase Amounts", x = "Purchase Amount", y = "Frequency")

# Task 6: Visualize Relationship (Scatter Plot)
ggplot(purchase_data, aes(x = CustomerID, y = PurchaseAmount)) +
  geom_point(color = "green") +
  labs(title = "Customer Purchase Amounts", x = "Customer ID", y = "Purchase Amount")

# Save the visualizations as image files
ggsave("D:/R Lab 2024/purchase_histogram.png", width = 8, height = 6)
ggsave("D:/R Lab 2024/customer_purchase_scatter.png", width = 8, height = 6)

```